

# Using Z-Wave Devices with Home Assistant on macOS Ventura (2017 iMac 5K)

**Introduction:** This comprehensive guide will show you how to set up Home Assistant on a 2017 iMac 5K running macOS Ventura, and integrate a Zooz “S7” (Zooz ZST10 700 Series S2) Z-Wave USB stick to control common Z-Wave smart devices. We’ll cover support for various device types (sensors, switches, locks, thermostats, etc.), step-by-step installation of Home Assistant natively on macOS, optional Docker deployment, connecting the Zooz stick (including any Ventura-specific driver or permission settings), and best practices for home automation and security. Clear instructions, tips, and troubleshooting steps are provided to ensure a smooth setup.

## Overview of Z-Wave Device Support in Home Assistant

Home Assistant’s Z-Wave integration (powered by **Z-Wave JS**) is compatible with all standard Z-Wave certified devices – you are not limited to any specific brand or model. **Any Z-Wave Plus controller, such as the Zooz 700 Series S2 stick, can communicate with Z-Wave sensors, switches, door locks, thermostats, and more from various manufacturers** <sup>1</sup>. All mains-powered Z-Wave devices in your network (e.g. plug-in modules, wired switches) will act as signal repeaters to strengthen the mesh network, regardless of vendor <sup>2</sup>. This means you can mix and match device brands freely; Home Assistant will interpret their signals via the Z-Wave stick’s radio.

### Common Z-Wave Device Types and Home Assistant Support:

| Device Type           | Home Assistant Entity Type                                                                | Notes on Setup and Use                                                                                                                                                                                                                |
|-----------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Smart Switch/<br>Plug | <code>switch</code> or <code>light</code> entity (toggle on/off, brightness if dimmer)    | Include like any Z-Wave device. Once added, you can control it via HA UI or automations. Mains-powered switches extend the Z-Wave mesh.                                                                                               |
| Light Dimmer          | <code>light</code> entity (with brightness control)                                       | Dimmers show up as lights with brightness sliders. They repeat signals (if not battery-powered) and support “ <b>multilevel switch</b> ” commands for smooth dimming.                                                                 |
| Door/Window<br>Sensor | <code>binary_sensor</code> (open/closed) and possibly <code>sensor</code> (battery level) | These battery-powered contact sensors report open/closed status. They appear as <code>binary_sensor</code> entities (often classed as door or window). Home Assistant will show their state changes for use in alarms or automations. |

| Device Type                                    | Home Assistant Entity Type                                                                               | Notes on Setup and Use                                                                                                                                                                                                                                                                                              |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Motion Sensor</b>                           | <code>binary_sensor</code> (motion detected/clear) and <code>sensor</code> (battery, illumination, etc.) | Motion or PIR sensors show as <code>binary_sensors</code> . Many multi-sensors also provide temperature, light, humidity, etc., as separate sensor entities. They may need a manual wake-up after inclusion to send all info.                                                                                       |
| <b>Smart Lock</b>                              | <code>lock</code> entity (lock/unlock) and sensors (battery, status)                                     | Z-Wave door locks are fully supported – you can lock/unlock from HA and see lock status. <b>Secure inclusion (S0/S2)</b> is required for locks. Home Assistant's Z-Wave JS handles S2 security (you'll enter the lock's DSK/PIN during inclusion). Once paired, locks can be controlled via HA or voice assistants. |
| <b>Thermostat</b>                              | <code>climate</code> entity (with temperature setpoints, mode, etc.)                                     | Z-Wave thermostats appear as climate devices, allowing you to read current temperature and adjust heating/cooling setpoints in HA. Some may also expose extra sensors (ambient temp, humidity). Include them like other devices; if they support S2 security, include securely.                                     |
| <b>Siren/Alarm</b>                             | <code>switch</code> or specialized alarm control (depending on model)                                    | Many Z-Wave sirens present as switches (on=alarm sound, off=silent). More advanced sirens might allow selecting tones via config parameters or a <code>siren</code> entity. You can incorporate them into security automations (e.g. trigger on intrusions).                                                        |
| <b>Multi-Button Remote or Scene Controller</b> | <i>No permanent entity</i> (triggers <b>events</b> for button presses)                                   | Handheld remotes or wall scene controllers often don't show up as entities. Instead, they send Z-Wave <b>Central Scene</b> events that Home Assistant can listen for in automations <sup>3</sup> . You can use the <b>Event</b> trigger in HA to react to button presses (e.g. to toggle lights).                   |

Home Assistant's Z-Wave JS integration supports both **Classic Inclusion** (putting the controller in add/inclusion mode and activating inclusion on the device) and **SmartStart** (where you scan a device's QR code *before* powering it on, and it joins automatically) <sup>4</sup>. SmartStart is available with 700-series controllers and SmartStart-capable devices – if you have newer devices with a QR code, Home Assistant allows you to scan it during inclusion for a simplified pairing process <sup>5</sup>. For classic inclusion, you'll use Home Assistant's interface to initiate adding a device, then follow the device's manual (e.g. pressing a button or sequence to include). We will cover the inclusion process in detail later.

**Security & S2 vs S0:** Modern Z-Wave devices (especially locks, garage controllers, etc.) often use Security S2 encryption, which is strongly preferred over the older S0. S2 provides encryption with minimal overhead, whereas S0 adds a lot of extra network traffic and can slow down your Z-Wave mesh <sup>6</sup>. Home Assistant will **automatically use S2** if both the controller and device support it, and will only fall back to S0 if necessary <sup>7</sup>. In practice, this means include **locks** and other security-critical devices with S2 whenever

possible – Home Assistant will prompt you to enter the device's 5-digit PIN (from the device's QR code or manual) during inclusion to authenticate S2. Older Z-Wave locks that only support S0 will still work (enter the network key to include securely), but be aware of the heavier traffic load with S0 <sup>8</sup>. We'll ensure the proper keys are set up for secure inclusion in the configuration steps.

## Installing Home Assistant Natively on macOS Ventura

Running Home Assistant “natively” on macOS means installing it directly on the Mac's OS (as a program) rather than using a virtual appliance or dedicated device. On a 2017 iMac (Intel-based) with macOS Ventura, you can install **Home Assistant Core** – the Python-based application – and run it as a service. Below are step-by-step instructions:

**1. Prepare the System (Python Environment):** macOS Ventura does not include an up-to-date Python 3 by default, so first ensure you have Python 3.10+ installed. You can download the official Python installer from python.org or install via Homebrew (`brew install python`). Verify that running `python3 --version` in Terminal shows a 3.x version. It's recommended to create a Python **virtual environment** for Home Assistant to avoid conflicts with other Python packages <sup>9</sup>, though this is optional. To create a venv (if using the built-in `venv` module):

```
# In Terminal, create a folder for Home Assistant and a virtual environment
mkdir -p ~/HomeAssistant && cd ~/HomeAssistant
python3 -m venv venv          # create a virtual env named "venv"
source venv/bin/activate     # activate the virtual environment
```

Once activated (you'll see the env name in your shell prompt), you're ready to install Home Assistant.

**2. Install Home Assistant:** Use pip to install the Home Assistant Core package. In Terminal, run:

```
pip install homeassistant
```

This will download and install Home Assistant and its dependencies. The process may take a few minutes. *(If you see any errors during installation, ensure that you have the Xcode command-line tools installed by running `xcode-select --install`.)* After a successful install, you will have the `hass` command available in your Python environment <sup>10</sup>.

**3. Initial Launch and Onboarding:** Start Home Assistant by running:

```
hass
```

The first startup will take several minutes as Home Assistant builds the initial configuration and downloads some default integrations. You'll see log output in the Terminal. Wait until you see messages indicating that Home Assistant is running and serving on port 8123. By default, it runs on **http://localhost:8123** <sup>11</sup> .

- **First-Time Setup:** Open a web browser on the iMac and navigate to `http://localhost:8123` . You should see the Home Assistant onboarding screen. Follow the prompts to create an admin user account, set your location/time zone, and optionally enable telemetry. Once completed, you'll reach the Home Assistant dashboard (which will be mostly empty until we add integrations/devices).
- **Allow Network Access (optional):** If you want to access Home Assistant from other devices on your network (e.g. your phone or laptop), make sure the Mac's firewall allows port 8123 or that you configure Home Assistant to listen on all interfaces. By default, Home Assistant should bind to all network interfaces. You can test by visiting `http://<YourMac'sIP>:8123` from another device. If it doesn't connect, check macOS Firewall settings in **System Settings > Network > Firewall** and add a rule to allow incoming connections for Python or Home Assistant.

**4. Running Home Assistant in the Background:** When launched via Terminal, Home Assistant will run as long as that session is open. To avoid having to start it manually each time, you can set it up to run in the background or on boot: - **Launch Daemon/Agent:** Home Assistant provides a convenience script for macOS to set it up as a Launch Daemon. You can run `hass --script macos install` (in your virtualenv) to register Home Assistant to start at boot and run as a background service. This will create a plist in `~/Library/LaunchAgents` to auto-start Home Assistant on login. If that script doesn't work on Ventura or you prefer manual setup, you can create a Launch Agent plist file yourself pointing to the `hass` executable. - **Prevent Sleep:** Since you're running on an iMac, you likely plan to keep it on 24/7 as a server. Go to **System Settings > Battery > Power Adapter** and enable **"Prevent computer from sleeping automatically when the display is off."** This ensures your Mac stays running Home Assistant continuously (the display can sleep, but the machine stays awake to run automations). Also disable "Put hard disks to sleep" and verify no aggressive sleep timers are set. If the Mac does sleep, Home Assistant (and your Z-Wave network control) will pause until it wakes.

**5. Home Assistant Interface Basics:** Once Home Assistant is up, familiarize yourself with the sidebar. Key areas we will use: - **Settings > Devices & Services:** where you add integrations (like Z-Wave) and manage devices. - **Settings > System > Logs:** to view logs (useful for troubleshooting). - **Developer Tools:** for advanced tasks (seeing events, states, etc., which can help in automation creation or debugging).

*Tip:* Although we focus on native installation, be aware there's an alternative one-click method: an unofficial **"Home Assistant Server" app on the Mac App Store** that packages Home Assistant OS in a lightweight VM. Some users opt for it for convenience (it costs a few dollars) <sup>12</sup> <sup>13</sup> . If you prefer a simple GUI approach and don't mind using a bundled VM, that app can automate much of the setup. However, the native install we just did is free and gives you full control and easy access to Home Assistant's files. Now, with Home Assistant running, we'll proceed to configure Z-Wave by connecting your Zooz USB stick.

## Setting Up the Zooz S7 Z-Wave USB Stick on macOS

The Zooz "S7" stick (Zooz ZST10 700 Series S2 USB stick) will serve as the Z-Wave radio controller for Home Assistant. Setting it up on macOS Ventura is straightforward since **no special driver is required** – it uses the built-in USB serial drivers on macOS <sup>14</sup> . Here's how to get it connected and recognized:

**1. Plug in the Zooz Z-Wave Stick:** Insert the Zooz ZST10 700 stick into a USB port on your iMac. Use a USB 2.0/3.0 Type-A port on the back of the iMac (2017 models have 4 USB-A ports). **Avoid USB hubs if possible during initial setup**, or use a powered hub to ensure the stick has sufficient power. Many users also recommend using a short USB extension cable to position the stick a few inches away from the computer's chassis – this can reduce any interference from the Mac's electronics and improve radio range <sup>15</sup> (Z-Wave operates at ~908 MHz, which isn't as susceptible to USB 3.0 interference as 2.4 GHz devices, but spacing out the radio is still good practice).

**2. Confirm macOS Recognizes the Stick:** macOS Ventura should detect the stick automatically as a USB modem device. You can verify this in a couple of ways: - **System Information:** Open **Applications > Utilities > System Information**, and look under **USB**. You should see an entry for a device (possibly labeled with a generic name or chipset). For example, it might appear as “*Silicon Labs CP210x*” or a CDC device. The Zooz 700 stick uses the standard USB CDC (Communications Device Class) interface, so it could show up as a modem. - **Network Preferences:** macOS might prompt you with a message like “New network interface detected” for the USB modem. If you go to **System Settings > Network**, you may see an entry for a device (e.g., “usbmodem14101” or similar). You **do not need to configure this as an internet connection** – you can ignore the network settings. The presence of this interface means macOS has recognized the USB stick. According to Zooz, on Mac “**the device will display as `/dev/cu.usbmodem` under Network settings**” <sup>16</sup>. In Ventura's Settings UI, it might show up with a slightly different name (including numbers). This is the serial port Home Assistant will use.

**3. Find the Device Path:** Home Assistant needs the path to the serial port for the Z-Wave stick. On macOS, serial devices appear as files under `/dev`. Open Terminal and list the devices by running:

```
ls /dev/cu.*
```

(the “cu” devices are the ones used for serial communications on Mac). You should see something like `/dev/cu.usbmodemXXXX` (the suffix can be a mix of numbers/letters). That is your Zooz stick. For example, it might be `/dev/cu.usbmodem141101` or `/dev/cu.usbmodem001`. Note this full path for use in Home Assistant. (You might also see a similar `/dev/tty.usbmodemXXXX` – typically you'll use the “cu” version for initiating connections.)

*No driver installation is needed for Zooz sticks on Mac.* The stick uses Apple's built-in driver (present since OS X), so once it shows up as a device, you're set <sup>14</sup>. If it doesn't appear at all in `/dev/`, try unplugging and replugging the stick, and ensure the LED (if any on the stick) lights up. Also, check that the USB port is functioning and not a power-only port.

**4. Permissions and Access:** Usually, your user account on macOS can access USB serial devices without extra configuration. Unlike Linux, there's no `dialout` group to configure. However, the device node's permissions can be checked by `ls -l /dev/cu.usbmodemXXXX`. It should be accessible to system users. If you run Home Assistant as your own user (as we did in the venv), it should be able to open the port. If Home Assistant is unable to access the port, you might need to run it with elevated privileges or adjust ownership, but this is rarely an issue on Mac. (If using Docker, we'll cover container access to USB in the Docker section.)

At this stage, the hardware is connected and recognized by the OS. Next, we will configure Home Assistant's Z-Wave integration to use this stick and set up the Z-Wave JS server that manages the network.

## Configuring the Z-Wave Integration in Home Assistant

Home Assistant uses a component called **Z-Wave JS** to interface with Z-Wave networks. The integration has two parts: a **Z-Wave JS Server** (which runs separately and communicates with the USB stick), and the **Home Assistant Z-Wave integration** which connects to that server. In Home Assistant *OS/Supervised* installations, the server runs as an add-on. In our macOS manual setup, we need to run the Z-Wave server ourselves (since we don't have the add-on system). We'll set it up and then add the integration in Home Assistant.

**1. Install/Run the Z-Wave JS Server:** The easiest way to run the Z-Wave JS Server (with a nice control UI) on a Mac is via Docker. (If you prefer not to use Docker at all, you would have to install Node.js and run the Z-Wave JS UI app from source, which is complex. Docker is recommended by the developers in this case <sup>17</sup> <sup>18</sup>.) We'll outline the Docker method here, but if you absolutely cannot use Docker, you could run Home Assistant in a VM with Home Assistant OS or attempt a manual Node.js setup.

**Using Docker for Z-Wave JS UI:** Ensure you have Docker Desktop installed on macOS. Then, you can launch the Z-Wave JS UI container (formerly known as zwavejs2mqtt, but it now encompasses the primary Z-Wave JS server plus an optional MQTT interface and web UI). Open Terminal and run:

```
docker run -d \
  --name zwave-js-ui --restart unless-stopped \
  -p 8091:8091 -p 3000:3000 \
  -e TZ=America/Toronto \
  -e SESSION_SECRET=mysecret \
  -v ~/HomeAssistant/zwave-js-store:/usr/src/app/store \
  --device /dev/cu.usbmodemXXXX:/dev/ttyACM0 \
  zwavejs/zwave-js-ui:latest
```

Let's break down this command: - It downloads/runs the `zwavejs/zwave-js-ui` image (latest version). - Maps port **8091** (the web management UI) and **3000** (the Z-Wave JS WS server) to your Mac. - Sets timezone (optional) and a session secret for the UI. - Binds the USB device: we map our Mac's `/dev/cu.usbmodemXXXX` to the container's `/dev/ttyACM0`. Inside the container (which runs Linux), the stick will be accessible as `/dev/ttyACM0`. - Data is stored in `~/HomeAssistant/zwave-js-store` on the Mac (adjust path as needed). The container will keep network info and config there.

Once this container is running, it is the Z-Wave JS server. You can open `http://localhost:8091` in a browser to access the Z-Wave JS UI panel. On first load, it will ask you to set a username/password for the admin interface. Then under **Settings > Z-Wave**, set the **Serial Device** to `/dev/ttyACM0` (which we mapped) and hit Save. It should connect to the stick and indicate your controller (showing something like "Zooz ZST10 700" with its Node ID 1). Also in settings, ensure "WS Server" is enabled on port 3000 (it usually is by default). This WebSocket server is what Home Assistant will talk to.

*Alternative:* If not using Docker, the process would be: install Node.js, download the Z-Wave JS UI release, and run it manually. This is beyond the scope of this guide, but know that it's possible (just more difficult to maintain). Docker container is the recommended approach for non-HAOS installs <sup>19</sup> <sup>18</sup> .

**2. Add the Z-Wave Integration in Home Assistant:** Now that the Z-Wave JS server is running, we integrate it with Home Assistant: - In Home Assistant's web UI, navigate to **Settings > Devices & Services**. - Click **"Add Integration"** (the + button) and search for "Z-Wave" in the list. Select **"Z-Wave JS"** (it may just show as "Z-Wave" with the logo). - Home Assistant will start the integration setup. Since we're not using Home Assistant's own add-on, **when it asks about the Z-Wave JS add-on, opt to "Install Manually" or uncheck any option to use the Supervisor add-on** (that option only applies to HA OS/Supervised) <sup>20</sup> <sup>21</sup> . Then it will prompt for the Z-Wave JS Server connection. - **Enter the WebSocket URL** for the server. If you ran the Docker as above on the same machine, it would be `ws://localhost:3000`. (If Home Assistant is in a container and Z-Wave JS in another, you'd use the appropriate host name or container name and port; on our native setup, localhost and port 3000 is correct.) Submit this info. - Next, it may ask for the **network key** if you haven't set up any. Since we already have Z-Wave JS UI running, it likely generated keys for S0 and S2 for us. If Home Assistant asks, you can leave fields blank to let it **generate keys automatically** <sup>22</sup> . In fact, Home Assistant or Z-Wave JS will ensure keys exist. The network keys (especially S0) are important for secure inclusion of older devices <sup>23</sup> . Home Assistant will generate unique 16-byte hex keys if none are provided <sup>22</sup> . You can later view or change these in the Z-Wave JS UI or Home Assistant configs. **Be sure to save these keys in a secure place** if you plan to back up or migrate – you'll need the same keys to seamlessly move your stick to another system or to re-include secure devices <sup>24</sup> <sup>25</sup> . - After submitting, Home Assistant will connect to the Z-Wave JS server. If successful, you'll see a message that the integration was added. In the Devices & Services list, **Z-Wave JS** will show up with 1 device (the Z-Wave controller itself). You can click it to see the controller info. This confirms Home Assistant is now linked to your Zooz stick via the Z-Wave server.

*Troubleshooting:* If Home Assistant cannot connect: - Double-check the WS URL. If using Docker on Mac, `localhost:3000` works because the container shares network via Docker Desktop proxy. If using another host or VM, use that IP/hostname. - Make sure no firewall is blocking that port 3000. - Ensure the Z-Wave JS UI container shows "Listening on 0.0.0.0:3000" in its logs and that it's running. - If you accidentally left the "use add-on" option checked, the integration might be waiting for an add-on that doesn't exist. Remove the integration and re-add, being careful to select manual setup.

**3. Verify the Controller in Home Assistant:** In HA, go to **Settings > Devices & Services**, find Z-Wave JS, and click **"Configure"**. You should see info about your Z-Wave stick (home ID, node ID 1, name of the controller). If you open the Z-Wave JS UI (at port 8091) concurrently, you'll see the same controller listed. Now you're ready to add devices to your Z-Wave network.

**Note:** The Z-Wave network itself (pairings of devices) is stored on the stick's memory (NVM). Home Assistant, however, will interview devices and cache additional info (like names, entity mappings) in its own config. On first setup, since our controller has no devices yet, there's nothing to interview. When you do add devices, if you ever restart Home Assistant, it will re-interview devices to get their state (it won't lose them, it just queries them on startup to sync state) <sup>26</sup> . For battery devices, this means they might show as "initializing" until their next wake-up. You can speed it up by manually waking them after a HA restart <sup>26</sup> .

Now we have Home Assistant and Z-Wave JS all set. Next, we'll add various Z-Wave devices and discuss any special procedures for each type.

## Including Z-Wave Devices: Sensors, Switches, Locks, Thermostats, etc.

Adding (including) devices to your Z-Wave network is done via the Home Assistant interface (or Z-Wave JS UI). Home Assistant provides a guided inclusion process. The general steps to include any device are:

1. **Initiate Add Device (Inclusion Mode):** In Home Assistant, go to **Settings > Devices & Services > Integrations > Z-Wave JS (Configure)**. Click **"Add Device"**. This will put the Z-Wave controller into *inclusion mode* (listening for new device join requests) <sup>27</sup>. Alternatively, you can click the **Add Integration -> Z-Wave** shortcut (My Home Assistant link) if available <sup>28</sup>. The UI will indicate it's searching for new devices.
2. **Activate Device Inclusion:** Now follow your device's manual to put it into inclusion/pairing mode. This often involves pressing a button or toggle on the device:
  3. *Battery sensors (door/window, motion):* usually have a small pinhole or button to press 3 times quickly, or a specific inclusion button.
  4. *Plug-in switches/dimmers:* often a button tap or long-press.
  5. *In-wall switches/dimmers:* toggling the switch on/off a certain way or pressing a config button.
  6. *Locks:* typically require entering a programming code or pressing a "learn" button. For instance, Schlage locks have a sequence of entering a programming code then '0' to include.
  7. *Thermostats:* often a menu option or special button combination to include.

Ensure the device is **near the controller** (at least for secure inclusion, being within a few feet is recommended for locks). When the device is triggered to include, it will send a node add request.

1. **Secure Inclusion Prompts:** If the device supports **S2 security**, Home Assistant will pop up a dialog asking for the **PIN or QR code**. Enter the 5-digit PIN from the device's QR label (usually on the device or in the manual). This is part of S2 authentication. Submit the PIN; Home Assistant will complete the secure key exchange. *(If the device supports SmartStart and you had scanned its QR earlier, you wouldn't need to do this live — it would join automatically when powered on. But for classic inclusion, entering the PIN is needed for S2.)* For **S0 devices**, no dialog appears because Home Assistant will use the S0 network key behind the scenes. Note that devices requiring S0 (like some older door locks) **must** be included with the network key or they won't function correctly <sup>29</sup> – Home Assistant/Z-Wave JS handles this as long as your network key was set.
2. **Inclusion Confirmation:** Watch the Home Assistant UI – it should report that a new device was added. Give it a moment to interview the device (the controller queries its supported features). You will then see the device appear in the **Devices** list under Z-Wave JS, and entities will start showing up. For example:
  3. A **switch** might create a `switch.lamp_switch` entity.
  4. A **multi-sensor** might create `binary_sensor.motion_hallway`, plus `sensor.hallway_temperature`, `sensor.hallway_humidity`, etc.
  5. A **lock** creates a `lock.front_door` entity (with lock/unlock controls) and possibly sensors for battery level or door bolt status.



6. A **thermostat** appears as a `climate.thermostat` entity with controls for mode and temperature.

**If the inclusion fails or times out:** Don't panic – it's common with battery devices if they sleep too quickly. You can **cancel** the inclusion in HA and try again. If a device was partly added (you see a "Node X" with no info or it's stuck "Interviewing"), you can remove it (exclusion) and re-add. See below for exclusion steps.

1. **Naming and Location:** After inclusion, go to **Settings > Devices & Services > Devices**, find the new device (it may have a generic name like "Zooz Unknown: id=xxxx" initially). Click it, then click the **pencil icon** to rename the device to something friendly (e.g. "Kitchen Motion Sensor"). You can also assign it to an Area (room) in Home Assistant which helps organize your entities. Naming devices in Home Assistant does *not* affect the Z-Wave network itself, but it will make your frontend and automations much easier to manage.
2. **Battery Device Tip:** For battery-powered devices, **wake them up a few times after inclusion** so that Home Assistant can finish interviewing and get all info (like supported sensors, configuration options). Waking up usually means pressing the inclusion button once (check the manual). Until a device is fully interviewed, some entities may not appear yet or it may show as "unknown device" manufacturer. Waking it ensures Home Assistant receives all details. You'll know it's fully interviewed when the device has proper name/manufacture info and all expected entities.

### Special Considerations by Device Type:

- **Sensors (Door/Window, Motion, etc.):** These often use **sleepy** operation to conserve battery. They wake up on a schedule (e.g. every 4 hours) to report in. If you change a configuration (like motion timeout), you must wait for or trigger a wake-up to send that config. When included, Home Assistant will list their state as on/off (open/closed, motion/clear). You can use these in automations (e.g. turn on lights when motion detected). Many sensors also send a **battery level** which shows up as a sensor in HA. Note that *some sensors do not automatically update their status in HA if manually triggered repeatedly in short succession* – they might have a timeout or send a single "active" report and then an "all clear" after some time. This is normal for motion sensors (to avoid spamming). You can adjust settings in the device (via Z-Wave config parameters accessible in HA's device config or Z-Wave JS UI) if needed to fine-tune behavior.
- **Lights and Switches:** These are typically straightforward. Once added, you can control them from Home Assistant UI or via voice (if you integrate with Siri/Alexa/etc). If a switch supports **dimming**, it will appear as a Light entity with a brightness slider. Home Assistant will automatically categorize many known Z-Wave devices (if the device database recognizes them). If a device appears just as a generic switch but you expected a light, you can change the entity's domain by using a customization (or just treat it as a switch – it's functionally similar). Most Z-Wave light switches report their physical state back to the controller (if you toggle the switch by hand, Home Assistant will get an update). If you encounter a switch that doesn't update when manually toggled, it might be an older model that lacks instant status – the workaround could be to enable polling for that node (not ideal, but possible – see Troubleshooting). Generally, **new Z-Wave Plus switches report state instantly via the Lifeline association**, so you should see near real-time updates in HA.
- **Door Locks:** When including a Z-Wave lock, it's best to have the lock close to the USB stick. Z-Wave locks often use a special low-power inclusion mode for security, which means if they're too far, the

secure key exchange might fail. If your lock is already installed on a door far from the iMac, consider temporarily moving the iMac (or use a long USB cable on the stick) closer for pairing, or pair the lock before mounting it. During inclusion, after entering the S2 PIN, wait patiently – locks can take a bit to securely exchange keys. Once included, you'll get a `lock.<name>` entity. You can lock/unlock from HA, include it in dashboards, and in automations (e.g. lock doors at 10 PM automatically). Home Assistant will also show the lock's status (locked/unlocked) and battery. You might also see a sensor for "alarm level" or "alarm type" on some locks – those are device-specific and indicate things like bad pin attempts or door tamper alarms. You can use **KeyMaster** (a third-party integration) if you want an easier UI to manage lock PIN codes via Home Assistant, since by default changing user codes requires sending Z-Wave config parameters.

- **Thermostats:** Z-Wave thermostats integrate as **climate** entities. After inclusion, you can set modes (heat/cool/auto/off depending on device) and adjust the target temperature. If the thermostat supports fan modes or hold modes, those may appear as additional controls. The current ambient temperature is usually provided by the thermostat as a sensor or as part of the climate entity. You might have separate entities for things like humidity or battery (if it's a battery-operated thermostat). One thing to note: some thermostats (battery ones) wake up periodically; if you send a setpoint change from HA, it might not take effect until the device wakes (it should happen within a few minutes if wakeup is configured). Mains-powered (or C-wire powered) thermostats are always listening and updates should apply immediately.
- **Others (Sirens, Garage Door Openers, etc.):** These devices each have quirks. Sirens usually behave like switches as mentioned. Garage door openers (using the Barrier Operator command class) appear as **cover** entities in Home Assistant (with open/close controls). For example, a Z-Wave garage controller will have a `cover.garage_door` entity. Home Assistant's Z-Wave JS supports the barrier operator class, allowing you to open/close and see door status. If you include one, make sure to test that it reports the correct status. Secure inclusion is usually required for these as well.

**SmartStart Inclusion:** If you buy new Z-Wave devices that support SmartStart (common with 700 series devices), you have the option to use SmartStart: In Z-Wave JS UI, you can go to **Control Panel > SmartStart** and add the device's DSK (by scanning QR code) before powering the device. Home Assistant's integration also offered a "Scan QR code" during add device <sup>4</sup>. If you scanned it and added, the device will join automatically within a few minutes of being powered on (no need to manually trigger inclusion). This can simplify inclusion for multiple devices – you could scan all their codes, then power them on one by one and watch them appear. SmartStart is optional; classic inclusion works for all devices.

**Exclusion (Removing a Device):** If you ever need to remove a device from your Z-Wave network (to reset it or to move it to a new controller), you should use the **Remove Device** (exclusion) function in Home Assistant *before* physically disposing or reusing the device. To do this, go to **Settings > Devices & Services > Z-Wave JS (Configure)** and click **"Remove Device"**, then **"Start Exclusion"** <sup>30</sup>. Then perform the device's exclusion procedure (usually the same as inclusion procedure – pressing the button, etc.). Home Assistant will confirm it removed a device if the signal went through <sup>31</sup>. This will factory reset that device's Z-Wave pairing, and it will disappear from your HA device list. If a device was never paired to your network and you can't add it, doing an exclusion first can help (it might have been paired to some other network out of the box). The Z-Wave stick can exclude **any** Z-Wave device (even one paired to a different hub) – this just resets the device's own memory of its network.

**Important Inclusion Tips:** - Do **not** use the button on the Z-Wave USB stick to include devices (if your stick has one). The Zooz 700 stick (S2) doesn't have an inclusion button at all (older sticks like Aeotec had one), so this may not apply. But in general, inclusion should be done through Home Assistant's interface. Including via the stick alone can cause Home Assistant to not know about the new node, and it can also lead to "phantom" nodes or broken routes <sup>32</sup>. Home Assistant's documentation explicitly says to avoid initiating inclusion from the stick directly <sup>32</sup>. - When including devices, **don't move the USB stick around the house**. In older Z-Wave practices, people took the stick to devices (using battery power on the stick) to include. This is not needed now and is discouraged <sup>32</sup>. Including in place (with a strong mesh or bringing the device near the hub temporarily) is the way to go. If your device is too far for initial inclusion, consider a portable inclusion (bringing the device closer) rather than moving the primary controller. Moving the controller can mess up routing tables because when you plug it back in, it's now far from where it was when devices were added. **Best practice:** include devices roughly in their intended location if possible, using mains repeaters as needed to extend range. - **Device Interview:** After inclusion, the Z-Wave JS integration does an interview process (querying supported command classes, versions, etc.). You can monitor this via the Z-Wave JS UI or Home Assistant logs. Wait until it's done (device shows as "Complete") before heavily using the device or creating automations, to ensure HA has all the info. If an interview fails (e.g., device battery died during it), you can re-interview the node later via the Z-Wave JS UI interface. - **Naming Devices:** It's helpful to name not only the device, but also customize entity names (especially if a multi-sensor provides generic names). You can do this in Home Assistant's **Entities** list or by clicking the entity and editing. For example, rename "Sensor 2 temperature" to "Living Room Temperature". This makes it easier to identify in the UI and scripts.

With devices now included, you can integrate them into Home Assistant scenes and automations (e.g. turn on Z-Wave lights when a Z-Wave motion sensor triggers, lock doors when a Zigbee button is pressed – any mix is possible since HA centralizes them).

## Home Automation Tips, Best Practices, and Optimizations

Setting up your devices is just the first step – now you'll want to leverage Home Assistant to create automations and ensure your smart home runs reliably and securely. Below are some best practices and tips for managing a Z-Wave network in Home Assistant, especially on a macOS-based setup:

- **Ensure a Strong Z-Wave Mesh:** Z-Wave is a mesh network, so the more **mains-powered nodes** you have, the more pathways signals have. If your iMac (with the Zooz stick) is centrally located, that's great. If not, consider adding a few plug-in outlets or repeaters in distant areas to relay signals. Remember, all non-battery Z-Wave devices act as repeaters by design <sup>33</sup> <sup>34</sup>. For example, if a door lock at the far end of the house is having connectivity issues, a Z-Wave smart plug halfway between can significantly help by forwarding commands. You can view the network topology in **Z-Wave JS UI's Network Graph** (it can draw a web of node connections) to identify weak spots.
- **USB Placement and Interference:** As mentioned, use a short USB extension cable to position the Z-Wave stick a short distance from the Mac. **Interference** can come from the Mac's USB 3.0 ports or other electronics. Many users have reported that moving the stick just a foot away on an extension resolves issues like devices failing to include or delayed responses <sup>15</sup> <sup>35</sup>. Also, keep it away from large metal surfaces (don't sandwich it between the iMac and a metal desk). Since the iMac 5K has a large aluminum body, having the stick on a short lead away from the back can improve their line-of-

sight to devices. Avoid coiling the cable with other cables which might cause EMI. Essentially, treat the stick like a little antenna that needs some breathing room.

- **Preventing macOS Sleep:** We touched on this, but it's worth reiterating: if the iMac goes to sleep, Home Assistant and the Z-Wave server will pause, meaning automations and device control won't work until it wakes. For a reliable home automation system, keep the Mac awake 24/7 (use display sleep only). You can also use the `caffeinate` command in macOS to temporarily prevent sleep when needed. Since this iMac might be a dedicated server, you could also set it to auto-login and start Home Assistant on boot for resilience after power outages or macOS updates.
- **Backup Your Home Assistant Config:** Regularly back up the Home Assistant configuration directory (by default, `~/.homeassistant` in a core install). This includes the entity registry and Z-Wave integration info stored by HA. While the Z-Wave network itself lives on the stick, having a backup of HA means if you migrate to another machine, you won't have to set up all your devices and names from scratch. Also, **backup your Z-Wave stick's NVM** if possible. The Z-Wave JS UI container has a feature to backup the controller's data (NVM backup). Use that to save a file of your network. In case the stick fails, this file can be restored to a new stick of the same series (700 series to 700 series) <sup>36</sup> <sup>37</sup>. Migrating between different series (700 to 800) is more complex but tools are evolving to support it <sup>38</sup> <sup>39</sup>.
- **Secure Your Home Assistant:** Since Home Assistant will be controlling locks and possibly alarm sensors, make sure you secure access to it. Use a strong password for your HA user, consider enabling 2FA. If you expose HA outside your network (for mobile access), use HTTPS (you can use Nabu Casa cloud or set up your own SSL via Let's Encrypt). On the Mac itself, keep it up-to-date and don't disable its firewall entirely (just allow HA's port). Also, **limit admin access** – if others in your home use HA, create non-admin users for them. This prevents accidental changes to critical settings.
- **Use Automations and Scenes:** Home Assistant's automation engine lets you tie together your Z-Wave devices with any other integrations. Some example ideas:
  - *Lighting automations:* Use motion sensors to turn on Z-Wave lights for a few minutes when motion is detected. Or use door sensors to trigger foyer lights on when a door opens at night.
  - *Security:* If you have Z-Wave door/window sensors on an alarm panel, you can create an alarm system in HA (or use the **Alarmo** integration for a ready-to-use alarm control panel). This can trip sirens (Z-Wave or otherwise) when intrusion is detected. You can also have your Z-Wave locks trigger an automation – e.g., if a lock is unlocked after 10pm, have HA send you a notification or turn on indoor lights.
  - *Lock User Codes:* If you install the custom **KeyMaster** integration, you get a nice UI to manage PIN codes for Z-Wave locks (schedule codes, one-time codes for guests, etc.). This is very handy for smart locks as it abstracts the Z-Wave Command Class for lock codes into an easy interface.
  - *Thermostat Schedules:* With HA you can bypass the limited scheduling in many thermostats and use HA to create daily schedules or integrate with presence. For instance, if no one is home (perhaps detected via a Z-Wave motion sensor or HA's Home/Away status), HA could set back the thermostat to save energy. In the morning, set a comfortable temp, etc. There are community plugins like Scheduler Card that can help create scheduling UI.

- *Energy monitoring:* If you have Z-Wave power plugs that report energy usage, you can track those in HA's Energy dashboard. This can help identify consumption patterns (e.g., how much power your Z-Wave space heater plug is drawing). Make sure to add the energy sensor to the Energy config.
- *Voice Control:* You can integrate HA with Google Assistant or Alexa (via Nabu Casa cloud or manual setup) – then all your Z-Wave devices can be voice-controlled. For example, “Alexa, lock the front door” or “Hey Siri, turn on living room lamp” (if using HomeKit integration).
- *Dashboards:* Create Lovelace dashboard views for your security status (locks and sensors), climate control, etc. The Mushroom or HomeKit Infused dashboard plugins (via HACS) have nice card designs for locks and lights.
- **Optimizing Z-Wave Performance:** Z-Wave is generally low-bandwidth but reliable. To keep it snappy:
  - Avoid unnecessary polling. By default, Z-Wave JS **does not poll devices regularly**, which is good <sup>40</sup> <sup>41</sup>. If you have an older device that doesn't report state (e.g., a legacy light switch that doesn't send updates), you *can* enable polling for just that device value in Z-Wave JS UI, but do so sparingly <sup>42</sup> <sup>41</sup>. Polling too often can flood the network and slow everyone down <sup>43</sup>. A better approach is event-driven: replace devices with Z-Wave Plus versions that report state, or use automations to request an update only at specific times (Z-Wave JS provides a `zwave_js.refresh_value` service for on-demand polling of a value) <sup>44</sup>.
  - Group commands when possible. If you have a scene that turns off 10 Z-Wave lights at once, consider creating a **Z-Wave scene** or using the multicast feature (Z-Wave JS supports multicast to send one command to multiple nodes). Home Assistant's UI doesn't directly expose multicast, but the `zwave_js.multicast_set_value` service is available for advanced use <sup>45</sup>. Alternatively, stagger the commands by a few hundred milliseconds in an automation if you notice any congestion.
  - Check the Z-Wave JS UI's **Statistics** for your nodes. It can show if certain nodes have a lot of transmission errors or retries. If a particular device has poor connectivity, addressing that (moving it or adding a repeater near it) will improve overall network health. The integration provides sensors for controller/node RTT and lost packets which can be useful for troubleshooting radio issues <sup>46</sup> <sup>47</sup>.
- **Firmware Updates for Devices:** Some Z-Wave devices allow OTA firmware updates through the controller. Z-Wave JS UI has support for OTA updates if you have the firmware files. For example, if Zooz releases a firmware update for your stick or any Zooz device, you can perform the update via the UI. Be cautious with firmware updates for the **stick itself**: Zooz notes that you should only update the stick's firmware if absolutely needed, as a failed update could brick the controller <sup>48</sup> <sup>49</sup>. The Home Assistant UI may warn if it detects your stick's firmware has known issues (for instance, early 700-series firmware had a bug causing dead nodes) <sup>50</sup> <sup>51</sup>. If you experience instability (e.g., the Zooz stick becomes unresponsive often), consider updating its firmware to the latest recommended version (Zooz provides a Windows tool for this <sup>52</sup> <sup>53</sup>). Otherwise, **“If it ain't broke, don't fix it”** applies – don't rush to update stick firmware just because a new one exists, unless it specifically addresses a problem you face <sup>49</sup>.
- **Using Integrations to Extend Functionality:** Home Assistant's power is in multi-protocol integration. Some helpful add-ons or custom integrations specifically for Z-Wave users include:

- **Z-Wave JS UI Add-on:** If you ever migrate to an installation of HA that supports add-ons (e.g., a VM with Home Assistant OS), you can use the Z-Wave JS UI as an add-on instead of Docker. It provides the same functionality within HA's Supervisor. <sup>54</sup> . The benefit is a slightly more integrated experience and easy updates, but functionally it's the same as what we set up.
- **Node-RED:** A popular add-on/integration for creating automations via a flowchart interface. You can use Node-RED (running as a separate app or add-on) to listen for Z-Wave events and create complex logic. Some users find it more flexible for complex automation than HA's built-in YAML automations. It's optional, but worth exploring if you like visual programming.
- **HACS (Home Assistant Community Store):** This allows installing community-made custom integrations and frontend plugins. For Z-Wave, you might find custom Lovelace cards like **Z-Wave Network Graph** visualizer or **battery level charts**. Also HACS has **KeyMaster** (for locks) and other tools.
- **Presence and Security:** Consider using multiple sensors to enhance reliability. For example, you might use a Z-Wave motion sensor plus a contact sensor together in an automation (so the light stays on while either is active). Or combine Z-Wave sensors with other tech (like an IP camera motion via AI) for verification. Home Assistant lets you create **template sensors** or **groups** (e.g., a group of door sensors that triggers if any open).
- **Voice and Remote Access:** As mentioned, leverage Nabu Casa or HomeKit integration to tie your Z-Wave gear into voice assistants. It adds a lot of convenience (e.g., tell Siri goodnight and have it lock doors and turn off lights via HA).

By following these practices, your Home Assistant-driven smart home should be robust and responsive. The Zooz stick and Home Assistant together provide a **local, no-cloud-required** system – which is great for privacy and reliability. (No dependency on SmartThings or any cloud, which was one motivation for many to move to HA <sup>55</sup> <sup>56</sup> .)

## Optional: Running Home Assistant (and Z-Wave) in Docker

As an alternative to the native installation, you may consider deploying Home Assistant in Docker containers – either now or in the future. Docker can simplify maintenance (upgrading is as easy as pulling a new image) and provides environment isolation. Here we outline how you might transition to a Docker-based setup on your Mac, while still using the Zooz USB stick.

**1. Home Assistant Container:** The official Docker image for Home Assistant (called **homeassistant/home-assistant**) contains Home Assistant Core. It does *not* include add-ons or the Supervisor, but since we can run other needed containers (like Z-Wave JS UI), it covers our needs. To run Home Assistant in Docker: - Install Docker Desktop on your Mac if you haven't. - Decide on a location for config. For example, create a folder `~/ha-config` to store Home Assistant config files. - Run the container:

```
docker run -d --name home-assistant --restart unless-stopped \
-e TZ=America/Toronto \
-v ~/ha-config:/config \
-p 8123:8123 \
--device /dev/cu.usbmodemXXXX:/dev/ttyUSB0 \
homeassistant/home-assistant:latest
```

This maps your Mac's `/dev/cu.usbmodemXXXX` (Zooz stick) into the container as `/dev/ttyUSB0`. We also open port 8123, set timezone, and mount the config volume. Home Assistant inside the container will see the Z-Wave stick as a serial device at `/dev/ttyUSB0`. When you add the Z-Wave integration in this setup, you would use that path (or better, the `/dev/serial/by-id/...` path inside the container if available – in Docker on Mac, the by-id might not persist, so `ttyUSB0` is okay if you only have one USB device). - The first run will create the config files in `~/ha-config`. Just as before, you'd then access HA at `http://<MacIP>:8123` and do onboarding. - **Migrating from native to container:** You can copy your existing config directory (`~/.homeassistant` from the venv setup) to the new `~/ha-config` before starting the container. This way all your settings, devices, and integrations carry over. Make sure to update any path references (for example, the Z-Wave JS server URL might change if you run it differently). - When running HA in Docker, if you also keep using the Dockerized Z-Wave JS UI we set up, note that now **both HA and Z-Wave JS UI are containers**. They can talk over the Docker network. You'd adjust the Z-Wave integration's WS URL to point to the Z-Wave container's address. If using Docker's default bridge network, you could connect them by name. A simpler way is to use `--network=host` for the Z-Wave JS UI container on Mac; however, Docker for Mac's host mode has limitations. It might actually be easier to run both containers on the same custom bridge network and use container names. For instance, start both `home-assistant` and `zwavejs` containers with `--network=home-net` (a user-defined network), then Home Assistant can connect to `ws://zwave-js-ui:3000` if the Z-Wave container is named `zwave-js-ui`.

The Docker approach is definitely more advanced, but it gives you a taste of how you could later move everything into containerized services. Many community guides suggest running Home Assistant, Z-Wave JS UI, and other services via Docker Compose on an always-on machine (like your iMac, Mac mini, or a Linux box) <sup>57</sup> <sup>58</sup>. This iMac is perfectly capable of doing so.

**2. Home Assistant OS in a VM (Alternative Expansion):** Another route for the future is to use a VM on the Mac to host Home Assistant OS, which includes the Supervisor and add-on store. This would give you an experience similar to running it on a Raspberry Pi or dedicated NUC, but using a VM on your iMac. You could use free tools like **UTM** or **VirtualBox** (note: VirtualBox on Apple Silicon is problematic, but on Intel iMac it works) to create a VM, attach the Home Assistant OS image, and passthrough the USB Z-Wave stick to it. This VM method might be appealing if you want the full Home Assistant supervised experience (with one-click add-on installs for things like Z-Wave JS UI, Node-RED, etc.), yet still run on your Mac. There are community guides for setting up HA OS on Macs (especially for Apple Silicon using UTM) <sup>59</sup> <sup>60</sup>, as well as using VMware or Parallels for Intel Macs <sup>61</sup> <sup>62</sup>. The process typically involves downloading the VHDX or QCOW2 image of Home Assistant OS and creating a VM with it.

In a VM scenario, you'd attach your Zooz USB stick to the VM (the hypervisor passes the USB through). The HA OS would then see it likely as `/dev/ttyACM0` or by-id, and you'd use the standard Z-Wave JS add-on inside HA. This is beyond the scope of this document, but keep it in mind if you later decide you want the convenience of Supervisor (for automatic updates and snapshots). Your iMac is powerful enough to spare a VM for this purpose.

**3. Docker Compose setup (optional):** If you get comfortable with Docker, using **docker-compose** YAML to define your Home Assistant stack makes it easy to manage. For example, a `docker-compose.yml` could define the `homeassistant` service, the `zwave-js-ui` service, an MQTT broker, etc., all with one command to bring up. This is how many "home server" setups are managed. There are plenty of examples online and even official references for running HA Container with Z-Wave JS UI in Docker <sup>54</sup> <sup>63</sup>.

**4. Pros/Cons of Docker on Mac:** One thing to note – Docker on macOS uses a lightweight Linux VM under the hood. There is a small performance overhead and USB passthrough is generally good, but if you notice any instability with the USB stick in Docker on Mac, the alternative would be running HA directly (which we did) or using a Linux environment. Most have found Docker on Intel Macs to be fine for Home Assistant and Z-Wave, but just be aware of that layer. In our steps above, we used container mapping to expose the USB device; that should work since Docker for Mac will handle connecting that through to the VM.

In summary, the **Docker deployment** route is a flexible expansion path. You might stick with the native Python install for now (especially if it's working well), but you have the option to containerize the setup to make upgrades and add-ons easier. Just remember, if you switch methods, always keep backups of your config and Z-Wave network NVM so you can migrate without re-including all devices.

## Troubleshooting Common Issues (macOS + Home Assistant + Z-Wave)

Even with a solid setup, you may encounter some issues. Here are common problems specific to this setup and how to address them:

- **Z-Wave USB Stick Not Detected by Home Assistant:** If Home Assistant (or Z-Wave JS UI) can't find the Z-Wave stick:
  - Ensure the stick is listed in `/dev/` as discussed. If not, try a different USB port or cable. On a 2017 iMac, all USB-A ports share a bus – if possible test with another device to ensure the port works.
  - If the device is listed (e.g. `/dev/cu.usbmodem...`) but Home Assistant can't access it, check permissions. Running Home Assistant as a normal user should be fine, but if you see errors about permission denied accessing tty device, you might need to adjust group membership. On macOS, serial devices often belong to the `wheel` or `tty` group. Ensure your user is an admin (admin accounts are essentially in the wheel group). You could also try running Home Assistant with elevated privileges as a test (`sudo hass`) to see if that resolves it – if it does, it's a permission issue; but rather than running HA as root (not recommended long-term), fix the dev node perms. You could create a udev rule on Mac via `etc/dev/` (macOS is not easily configurable in this way, though).
  - Check that no other application is grabbing the serial port. macOS might create a `/dev/tty.usbmodem` as well – in some cases, software might try to use the modem (though unlikely). If you opened the Network preferences and clicked "Apply" on the new modem interface, macOS might "occupy" it. Delete any configuration for the USB modem in Network settings or set it to inactive. This can free the port. One user noted that on Mac selecting the device in some apps gave a "Line in use" error <sup>64</sup> – meaning another process had it open. Ensure only the Z-Wave JS server is connecting to it.
  - If using Docker, double-check the `--device` flag. In Docker for Mac, the device path is from the perspective of the macOS host (which we did). Also ensure the container is running with `--privileged` or specific device access (we did provide the device). Missing that will definitely cause the container to not see the stick <sup>65</sup> <sup>66</sup>.
- Reboot the iMac if needed. Sometimes the USB stack in macOS can glitch and a device might not appear until a reboot (or power cycle of the device).
- **Inclusion Issues (Device not pairing):** If you put HA in inclusion mode but the device isn't found:



- Make sure the device is reset or not already paired to another hub. Perform an **exclusion** (even on a brand new device) by putting HA in remove mode and triggering the device. Often a device fresh out of the box might still need an exclusion if it was factory-tested.
- Bring the device closer for inclusion. Sometimes distance is the issue – secure devices especially have limited inclusion range by design. Once included, you can install them in their final location and they'll use mesh hops, but for inclusion try within a few feet of the stick.
- Watch the Z-Wave JS UI logs during inclusion (enable debug logging) – it might tell you if a secure inclusion failed (bad PIN, etc.). If S2 inclusion fails, you might see the device added insecurely (which for locks means it won't accept commands). In that case, exclude and re-include, paying attention to the PIN prompt.
- For locks: if inclusion fails repeatedly at S2, try S0 fallback (some locks let you choose a legacy pairing mode by not entering a PIN – in HA, if you skip the S2 PIN, it might try S0 if the device supports S0). S0 should only be a last resort (it has heavy overhead <sup>8</sup>), but it can get a difficult lock paired. After pairing with S0, you *might* see a slight battery drain or delay in lock responses due to the extra frames, but it should work.
- **Ghost Nodes:** If a pairing failed mid-way, you might end up with a “Node XX” that is incomplete. These are called ghost nodes. They can cause network issues (controller thinks an ID is taken when device isn't actually there). It's best to remove them: In Z-Wave JS UI, you will see the node with no info – select it and choose “Remove failed node”. You can only remove it if the controller deems it failed. You might have to mark it as failed first (there's an option for that). Once it's removed, the node ID becomes free for future devices. Always clean up ghosts, as they can otherwise make the controller try routing through a non-existent node.
- **Devices Not Updating Status or Seem “Offline”:** If a node shows as unavailable or doesn't update:
  - Battery devices may sleep. Check the last seen time. Trigger them physically to wake them and see if HA gets an update.
  - If a device is marked **Dead** (in Z-Wave JS UI it might show with a red status), that means the controller hasn't gotten a response after multiple tries. This could be due to range or the device being unplugged (for a plug module) or a power outage. Try **healing the node**: In Z-Wave JS UI or HA's Z-Wave config panel, there's usually a “heal node” or you can just wait – Z-Wave JS automatically does some self-healing. A “Heal Network” function can be invoked (in older Z-Wave it was common to do nightly network-wide heal; Z-Wave JS does not require manual healing as often, it updates routes on the fly, but if you moved devices, a heal can help).
  - If a node remains dead, power cycle that device (pull batteries or air-gap a switch, then restore power). It may come back.
  - For devices that *never* report state (older Z-Wave non-plus): consider setting up a **refresh automation**. For example, some older dimmers didn't send instant status – if you need HA to know when it's manually toggled, you could poll it every minute or so, or better, when some related event happens (like motion in the room, assume someone might have toggled the lamp). Use the `zwave_js.refresh_value` service as needed <sup>44</sup>. This is a hack and should be last resort due to the mentioned network traffic issues with polling <sup>43</sup>.
- Check associations: Ensure each device is associated to the controller's Node 1 in Lifeline. Z-Wave JS does this automatically, but if you migrated from a different controller, some associations might be off. Usually not an issue in fresh setups.

- **Home Assistant Slowdowns or Freezes:** If Home Assistant becomes sluggish or unresponsive:
  - Check the logs (HA logs and system logs). One bad integration (unrelated to Z-Wave) could hang the event loop. If Z-Wave is suspect, enable debug logging for `zwave_js` to see if it's spamming errors.
  - If you have a very large Z-Wave network (say 50+ nodes), on an Intel iMac this should still be fine. The iMac has plenty of CPU/RAM. But after a restart, interviewing many nodes can put load on the system. It's best not to restart HA during times you need immediate responsiveness; let it finish the device interviews first. You can stagger waking battery devices to expedite that process.
  - The Docker VM on Mac could also cause slowness if it's constrained. You can allocate more RAM/CPU to Docker in its settings if needed.
- **Z-Wave JS UI Access:** If you can't reach the Z-Wave JS UI at its address (localhost:8091 or similar):
  - Make sure the container is running. Use `docker ps` to list running containers.
  - If it's running but not reachable, perhaps the port is not published or you're using host networking which on Mac might not expose it to your Mac's localhost. We suggested `-p 8091:8091`, so that should map it. Alternatively, you can check Docker Desktop's list of containers and see if there's any error logs.
  - Remember, Z-Wave JS UI is optional for day-to-day use – you can do most things in Home Assistant's own interface now (like adding/removing devices, setting parameters via the Configure Device page, etc.). But the UI is very helpful for advanced management (backups, viewing logs).
- **Firmware Warnings:** Home Assistant might show a persistent notification if it detects your controller's firmware is in a problematic range (for example, a 700 series stick on SDK 7.19). This warning is informational. As Zooz support notes, unless you are *experiencing* issues, you can ignore it safely <sup>49</sup>. If you do want to update the stick firmware, you'll typically need a Windows PC (Simplicity Studio or the Zooz OTA tool) <sup>67</sup> <sup>68</sup>. It's a bit of a process (and somewhat risky). So weigh the need. The community often shares experiences; e.g., some users had improved stability after updating their Zooz 700 stick to firmware 7.17.2 or above to fix dead node issues <sup>69</sup> <sup>70</sup>. Check Zooz forums or Home Assistant community if you're unsure. And always backup NVM before firmware changes.
- **macOS Updates:** When you update macOS, always check that the Home Assistant service is still running afterward. A macOS update might reboot the Mac; if you set up LaunchAgent correctly, HA should start on login. If it didn't, you may need to log in and start it manually. Also after major macOS upgrades, re-verify that the USB stick is recognized (sometimes security settings might reset if, say, kext approvals were needed – but for our CDC device, it shouldn't be an issue).
- **Hardware Considerations:** The 2017 iMac is powerful and has the advantage of Ethernet (use Ethernet for the Mac's network connection if possible, rather than Wi-Fi, to ensure robust connectivity for any cloud integrations or remote access). The iMac does consume more power than a small hub like a Raspberry Pi; if you find it overkill to keep it on 24/7, you could eventually migrate Home Assistant to a low-power device. But many people repurpose older Macs as servers successfully <sup>71</sup> <sup>72</sup>. As long as the machine is stable, you can rely on it. Keep an eye on the iMac's temperature and set appropriate cooling (the HA workload is usually light, but if you also run Plex or other services as in some user cases <sup>73</sup> <sup>74</sup>, ensure it has ventilation).

By systematically addressing issues with these tips, you can usually resolve any hiccups in your Home Assistant Z-Wave setup. The combination of Home Assistant's detailed logs, Z-Wave JS UI's network tools, and the supportive HA community forums will help troubleshoot most problems. Remember, a lot of issues boil down to either **communication range** or **mis-configuration**. With the iMac and Zooz stick configured as above, you've eliminated many potential mis-configurations, so range and device quirks become the focus – which you can solve by adding repeaters or adjusting device settings as needed.

---

**Conclusion:** You now have a fully functional Home Assistant installation on macOS Ventura, integrated with a Zooz S2 700 Series Z-Wave USB stick, ready to manage a variety of Z-Wave devices. We covered installing Home Assistant natively, optionally moving to Docker, pairing common device types, and using best practices to optimize your smart home's reliability and security. With this setup, your 2017 iMac becomes a powerful home automation hub – controlling lights, securing doors, regulating temperature, and much more, all through Home Assistant's unified interface. Enjoy your smart home, and happy automating!

### Sources:

- Home Assistant Z-Wave integration documentation 1 32 6 15
- Zooz ZST10 700 USB Stick user manual 14 16
- Zooz support knowledge base on Home Assistant & Z-Wave firmware guidance 49
- Home Assistant Community and Reddit discussions on macOS installation and Z-Wave setup 17 18 69 .

---

1 2 3 4 5 6 7 8 15 20 21 22 23 24 25 26 27 28 29 30 31 32 35 40 41 42 43 44 45 46  
47 54 63 Z-Wave - Home Assistant

[https://www.home-assistant.io/integrations/zwave\\_js/](https://www.home-assistant.io/integrations/zwave_js/)

9 10 11 Home Assistant - A Comprehensive Guide to install Home Assistant on Your Computer

<https://mrvautin.com/a-comprehensive-guide-install-home-assistant-on-your-computer/>

12 13 HA Server on Apple App store : r/homeassistant - Reddit

[https://www.reddit.com/r/homeassistant/comments/1ijp6xr/ha\\_server\\_on\\_apple\\_app\\_store/](https://www.reddit.com/r/homeassistant/comments/1ijp6xr/ha_server_on_apple_app_store/)

14 16 33 34 22-3-3-zst10-700-manual.cdr

<https://www.getzooz.com/downloads/zooz-700-series-z-wave-stick-zst10-700-manual-online.pdf>

17 18 19 65 66 ZWavejs2Mqtt with Home Assistant Core (venv) - Z-Wave - Home Assistant Community

<https://community.home-assistant.io/t/zwavejs2mqtt-with-home-assistant-core-venv/397944>

36 37 38 39 48 49 Z-Wave Stick & Home Assistant: Troubleshooting and Support - Zooz Support Center

<https://www.support.getzooz.com/kb/article/1957-z-wave-stick-home-assistant-troubleshooting-and-support/>

50 51 69 70 Aeotec Zwave S2 stick to new 7 series - Z-Wave - Home Assistant Community

<https://community.home-assistant.io/t/aeotec-zwave-s2-stick-to-new-7-series/412070>

52 53 67 68 Z-Wave Controllers - Home Assistant

<https://www.home-assistant.io/docs/z-wave/controllers/>

55 56 Setting up Zigbee/Z-wave on a mac - Zigbee - Home Assistant Community

<https://community.home-assistant.io/t/setting-up-zigbee-z-wave-on-a-mac/442075>

57 58 71 72 73 74 Thoughts on Running Homeassistant with Z-Wave on a Mac Mini : r/homeassistant

[https://www.reddit.com/r/homeassistant/comments/bk9tqe/thoughts\\_on\\_running\\_homeassistant\\_with\\_zwave\\_on\\_a/](https://www.reddit.com/r/homeassistant/comments/bk9tqe/thoughts_on_running_homeassistant_with_zwave_on_a/)

59 [Guide] Home Assistant on Apple Silicon Mac using HA OS AArch64 ...

<https://community.home-assistant.io/t/guide-home-assistant-on-apple-silicon-mac-using-ha-os-aarch64-image/444785>

60 Home Assistant OS on macbook - Configuration

<https://community.home-assistant.io/t/home-assistant-os-on-macbook/767942>

61 How to install Home Assistant on Intel macOS for free! - YouTube

<https://www.youtube.com/watch?v=gMUvfLozon0&pp=0gcJCdgAo7VqN5tD>

62 Install Home Assistant Operating System macOS without VirtualBox

<https://community.home-assistant.io/t/install-home-assistant-operating-system-macos-without-virtualbox/353666>

64 akauppi/raspberry-pi-zwave-setup: [WIP] Setup of RPi4 + ... - GitHub

<https://github.com/akauppi/raspberry-pi-zwave-setup>